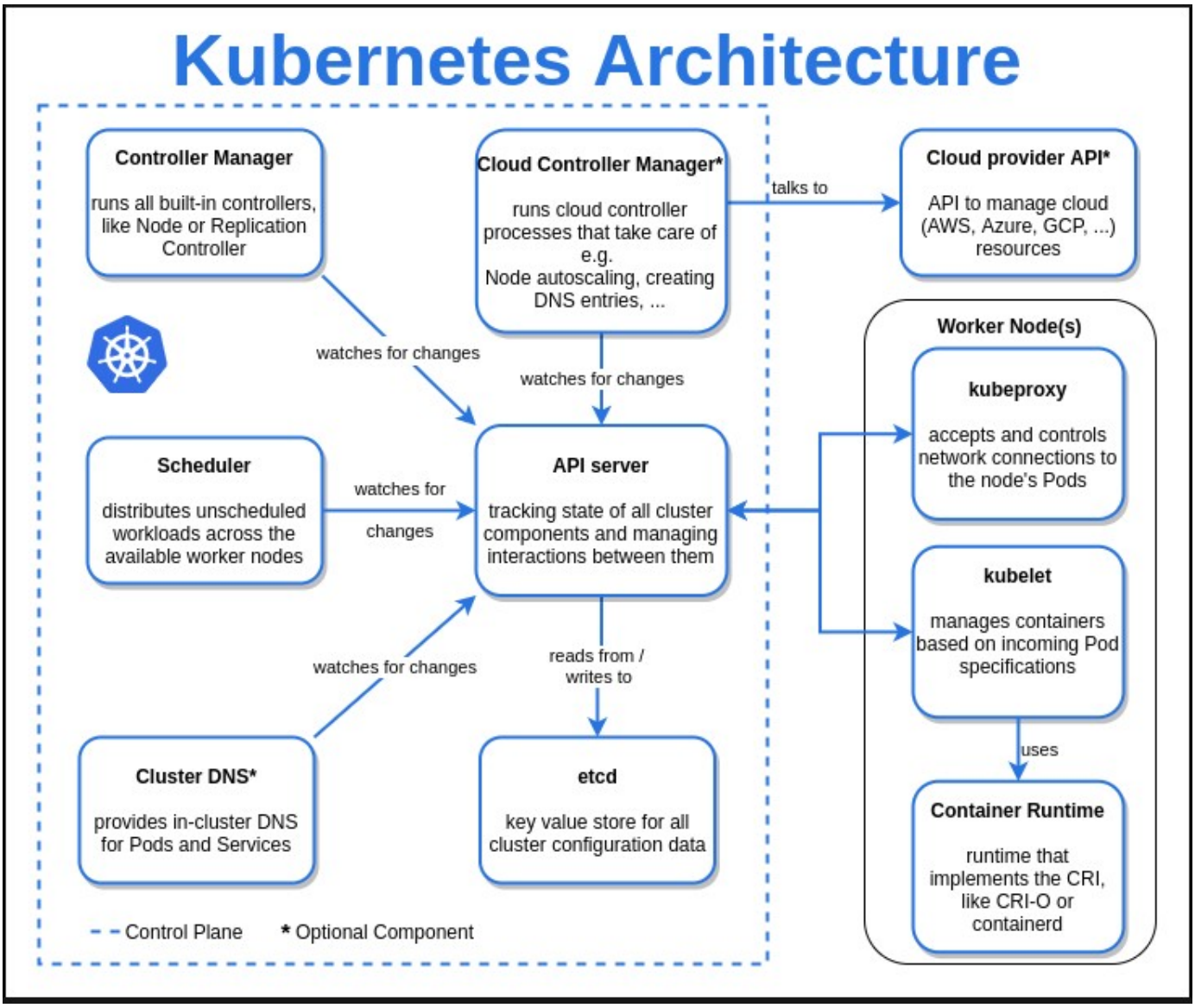
Light

PSScriptAnalyzer passing | Create Release passing | Check kubernetes App

LICENSE MIT | FORKS 0 | STARS 1 | CONTRIBUTORS 1 | ISSUES 2 OPEN

LINKEDIN

LEARNING KUBERNETES



Explore the docs »

Main Page - Code Page - Report Bug - Request Feature

Summary

▶ **TABLE OF CONTENT**

About Project

This project aims to help students or professionals to learn the main concepts of kubernetes

(back to top)

Getting Started

This is an example of how you may give instructions on setting up your project locally. To get a local copy up and running follow these simple example steps.

Prerequisites

This is an example of how to list things you need to use the software and how to install them.

- git
- Virtual Box and extension
- Vagrant

Installation

Clone repository

```
git clone https://github.com/marcossilvestrini/learning-kubernetes.git
```

Set ssh keys in security folder

```
# generate ssh key pair for your user access hosts
ssh-keygen -q -t ecdsa -b 521 -N '' -f ~/.ssh/id_ecdsa <<<y >/dev/null 2>&1
cp ~/.ssh/id_ecdsa.pub security/

# generate ssh key pair for rancher
ssh-keygen -q -t ecdsa -b 521 -N '' -f security/kubernetes-key-ecdsa <<<y >/dev/null
```

Set network

Set network configuration for each VM in Vagrantfile. Example:

```
# NETWORK
    ol9_server01.vm.network "public_network", nic_type: "virtio", mac: "080027f3066a
      "Intel(R) I211 Gigabit Network Connection",
      "MediaTek Wi-Fi 6 MT7921 Wireless LAN"
    ]
```

Light

Set VM's resources

Set resource configuration as CPU, Memory,etc for each VM in Vagrantfile. Example:

```
# PROVIDER
infra_server01.vm.provider "virtualbox" do |vb|
    vb.linked_clone = true
    vb.name =VM_INFRA_SERVER01
    vb.memory = 2048
    vb.cpus = 1
end
```

Up kubernetes cluster

```
cd learning-kubernetes/vagrant/linux
vagrant up
```

Usage

Use this repository for get learning about kubernetes exam

(back to top)

Roadmap

- [x] Create repository
- [x] Create github action for automation tasks
- [x] Install kubernetes cluster
- [x] Install kubectl
- [x] Add Examples kubernetes pods
- [x] Add Examples kubernetes deployment
- [x] Add app deployment
- [x] Create docker image with project contents
- [ ] Create github action for build docker image

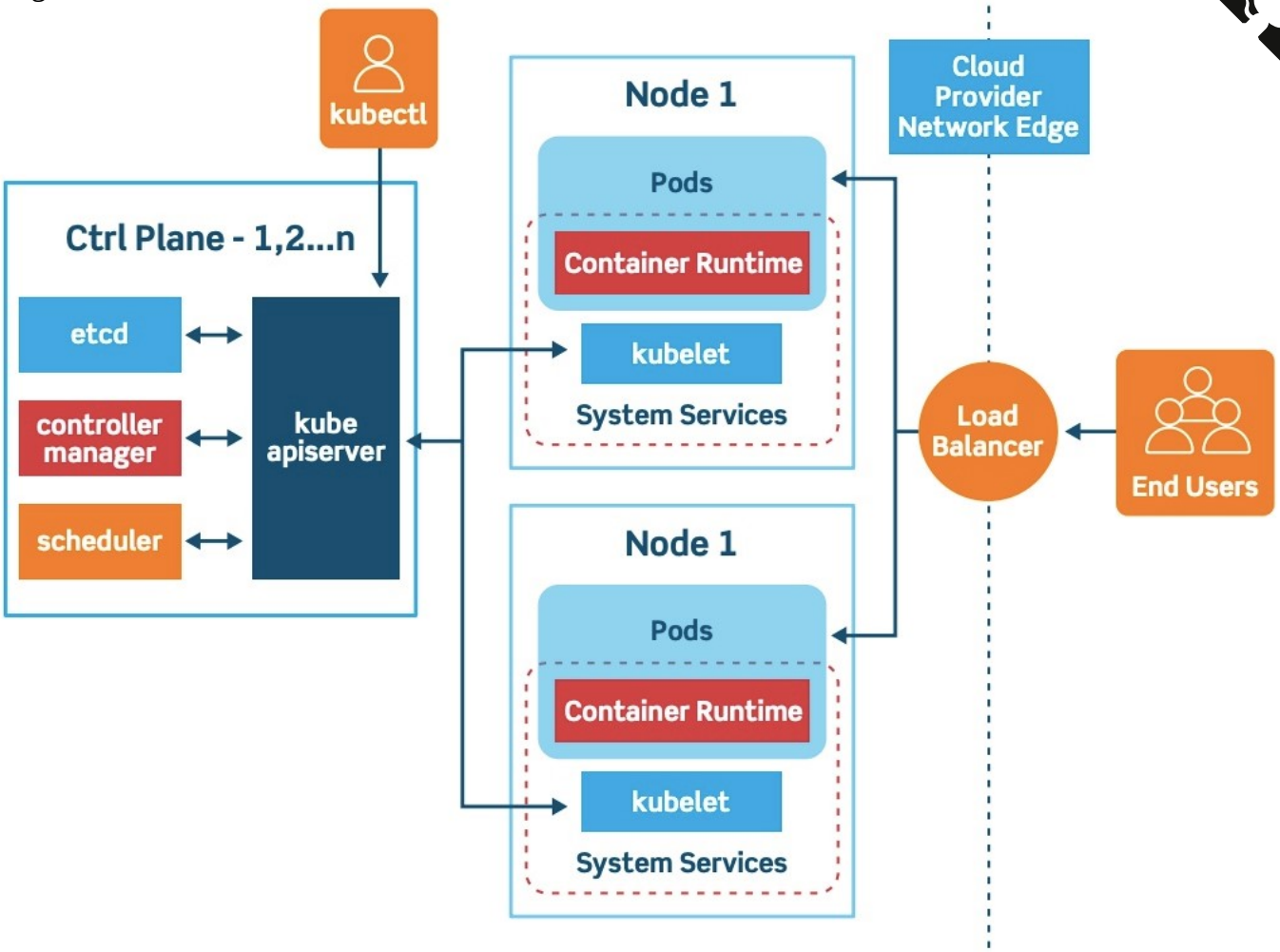
(back to roadmap)

(back to top)

kubernetes Engine work with namespaces(PID,NET,IPC,MNT,UTS) and cgroups.

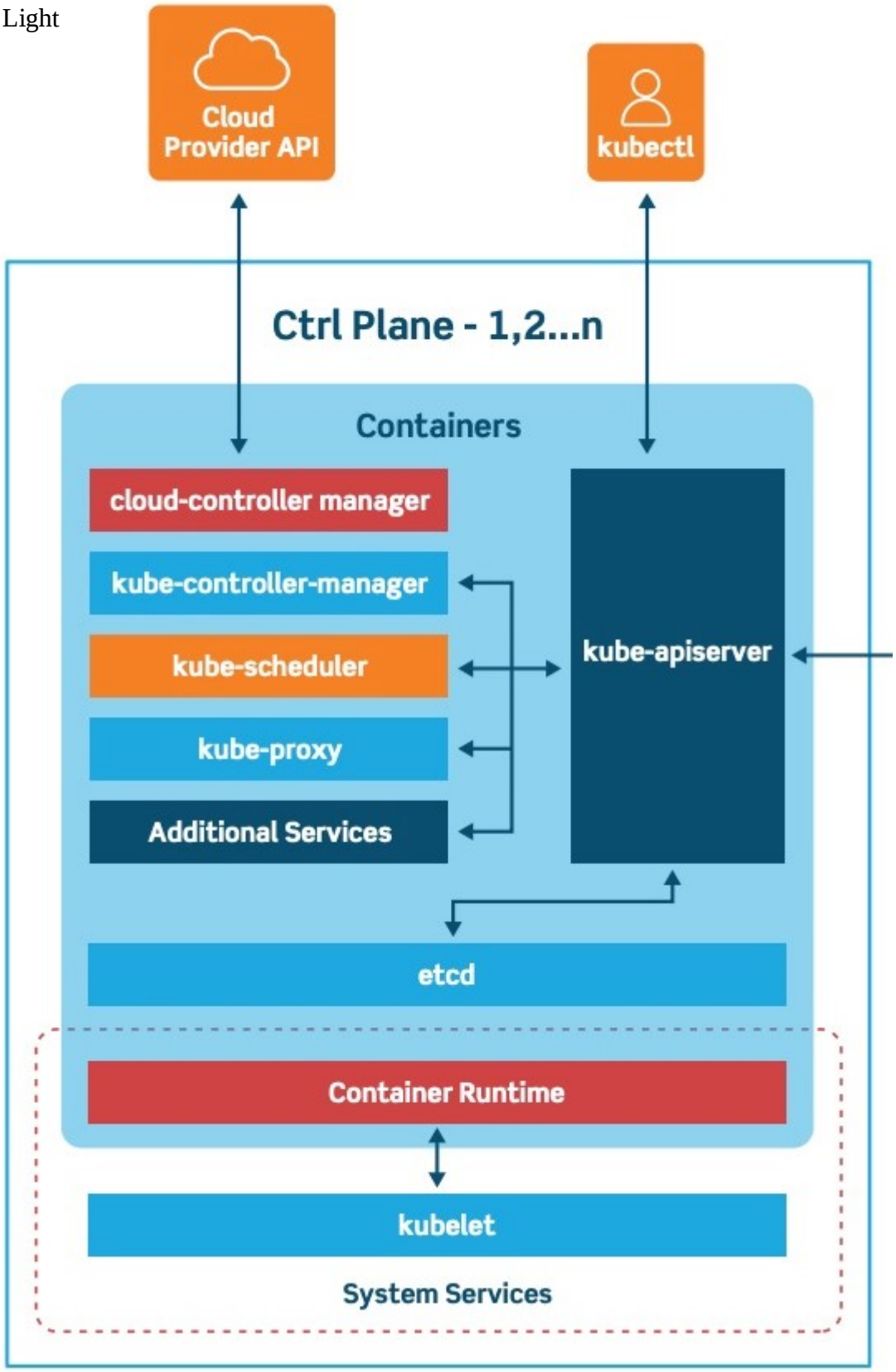
Kubernetes Architecture

Light



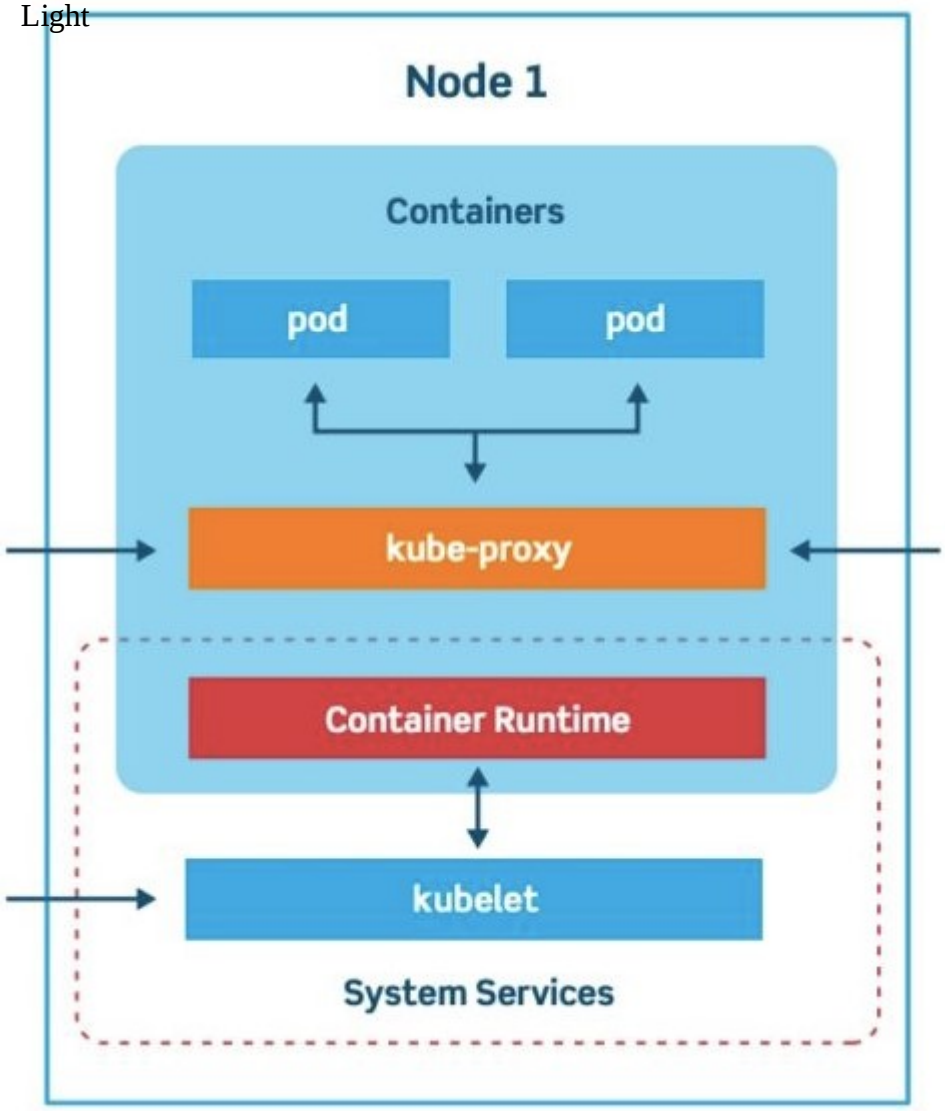
Control Plane

Light


Cloud Provider API
kubectl
Ctrl Plane - 1,2...n
Containers
cloud-controller manager
kube-controller-manager
kube-scheduler
kube-proxy
Additional Services
kube-apiserver
etcd
Container Runtime
kubelet
System Services

Node

Light



Kubernetes ports

CONTROL PLANE

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	6443*	Kubernetes API server	All
TCP	Inbound	2379-2380	etcd server client API	kube-apiserver, etcd
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	10251	kube-scheduler	Self
TCP	Inbound	10252	kube-controller-manager	Self

Light

WORKERS

Protocol	Direction	Port Range	Purpose	Used By
TCP	Inbound	10250	Kubelet API	Self, Control plane
TCP	Inbound	30000-32767	NodePort	Services All

Install kubernetes

Minikube

```
# install
curl -Lo minikube https://storage.googleapis.com/minikube/releases/latest/minikube-l
chmod +x ./minikube
sudo mv ./minikube /usr/local/bin/minikube

# get version
minikube version

# set hypervisor
minikube config set driver <YOUR_HYPERVISOR>

# up without hypervisor
minikube start --driver=hyperkit

# create cluster
minikube start --nodes 3 -p multinode-cluster

# get status of cluster
minikube status

# get ip address
minikube ip

# access minikube host
minikube ssh

# dashboard
minikube dashboard

# logs
minikube logs

# delete cluster
minikube delete
minikube delete --purge
```

Kind

```
# install
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.14.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind

# create cluster
kind create cluster
kind create cluster --name silvestrini

# get clusters
kind get clusters

# delete clusters
kind delete clusters $(kind get clusters)

## create yaml
cat << EOF > $HOME/kind-3nodes.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF

# create cluster
kind create cluster --name kind-multinodes --config $HOME/kind-3nodes.yaml
```

RKE2

For create kubernetes cluster using RKE2, see scripts in folder scripts/rke2

Some commands of stack rke2

Set your PATH variable:

```
export PATH=$PATH:/opt/rke2/bin:/var/lib/rancher/rke2/bin
```

CONTAINERD - ctr Commands

```
#list containers using ctr
ctr --address /run/k3s/containerd/containerd.sock --namespace k8s.io container ls
```

CONTAINERD - crictl Commands

```
#list containers using crictl
```

```
## example 1
export CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml
crictl ps

## example 2
crictl --config /var/lib/rancher/rke2/agent/etc/crictl.yaml ps

## example 3
crictl --runtime-endpoint unix:///run/k3s/containerd/containerd.sock ps -a


# stats containers
crictl stats
```

Logging

```
journalctl -f -u rke2-server
/var/lib/rancher/rke2/agent/containerd/containerd.log
/var/lib/rancher/rke2/agent/logs/kubelet.log
```

Reference: https://gist.github.com/superseb/3b78f47989e0dbc1295486c186e944bf

Kubectl

Install

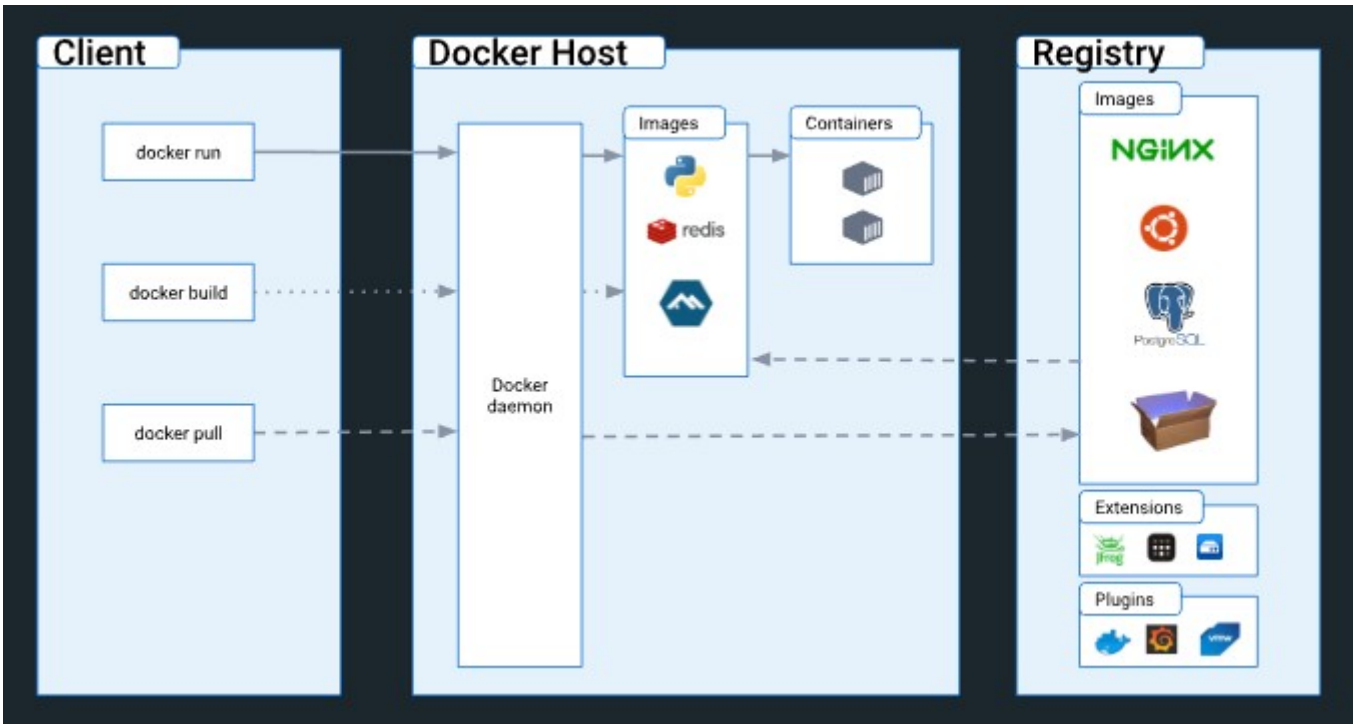
```
# install
curl -LO https://storage.googleapis.com/kubernetes-release/release/`curl -s \
https://storage.googleapis.com/kubernetes-release/release/stable.txt`/bin/linux/amd6
chmod +x ./kubectl
mv ./kubectl /usr/local/bin/kubectl

# get version
kubectl version  --output=yaml --client

# kubectl autocomplete
source <(kubectl completion bash)

# kubectl alias
alias k=kubectl
complete -F __start_kubectl k
```

Commands - Kubectl

```
########## resources ############
```

```
# list all resources
kubectl get all

########## namespaces ###########

# get namespaces
kubectl get namespaces

# describe namespaces
kubectl describe namespaces

########## nodes ############

# list nodes
kubectl get nodes
kubectl get nodes -o wide

# delete node
kubectl drain <node_name> --ignore-daemonsets --delete-emptydir-data
kubectl delete node <node_name>

# get logs
kubectl logs my-nginx
kubectl logs -f my-nginx
kubectl logs -n kube-system --all-containers=true etcd-control-plane01
```

Containers

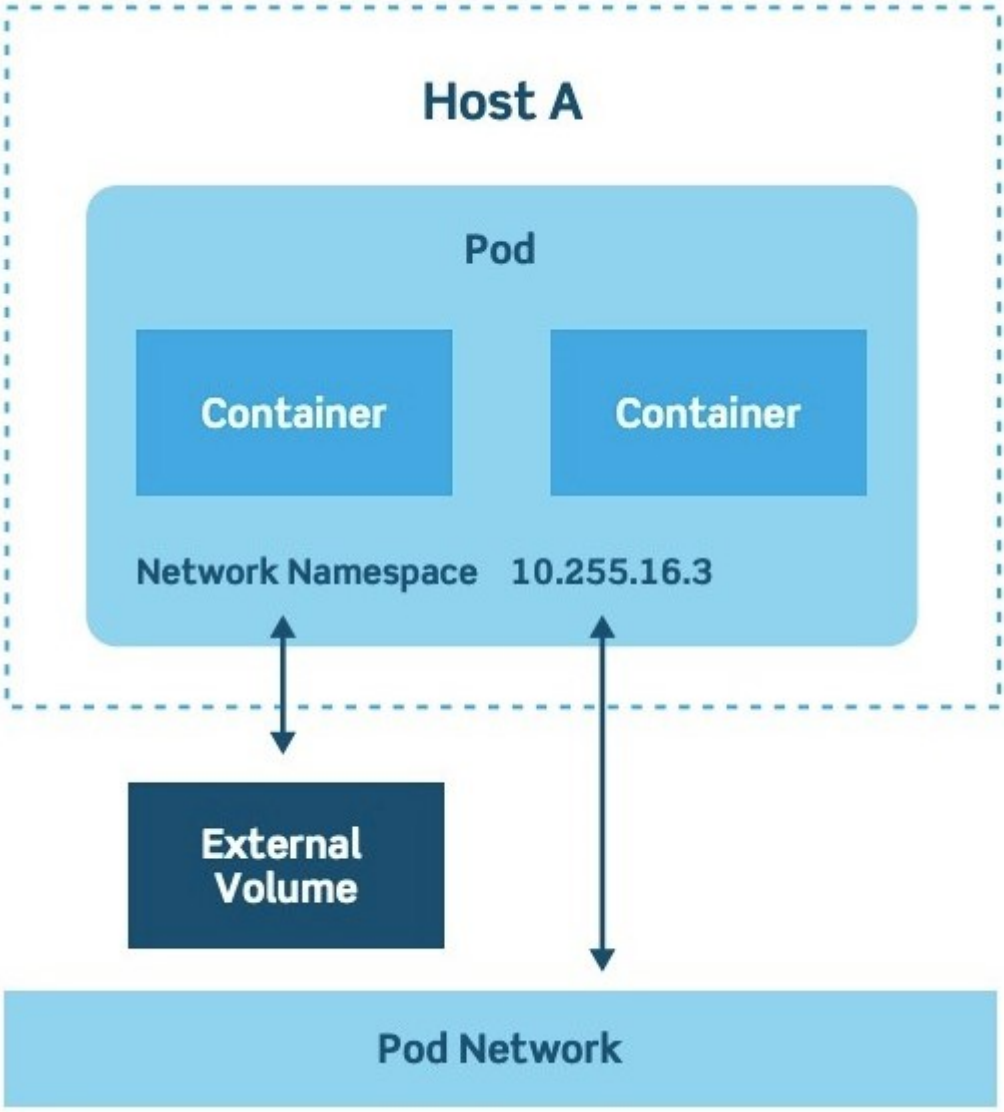


Commands - Containers

Light

```
# get containers in pod
kubectl -n kube-system  describe pods kube-proxy-worker01 | grep -ws -A 10  Contai

# connect in container
kubectl attach silvestrini -c infra

# connect in container
kubectl exec -it pod_name -c container_name bash
kubectl exec infra ls
kubectl exec silvestrini -c infra -- ls
kubectl exec silvestrini -c infra -it sh

# access container in specific namespace
kubectl exec -it -n kube-system  kube-proxy-worker01 -c kube-proxy -- bash
```

Pods



A pod is the smallest execution unit in Kubernetes. A pod encapsulates one or more applications. Pods are ephemeral by nature, if a pod (or the node it executes on) fails,

Light

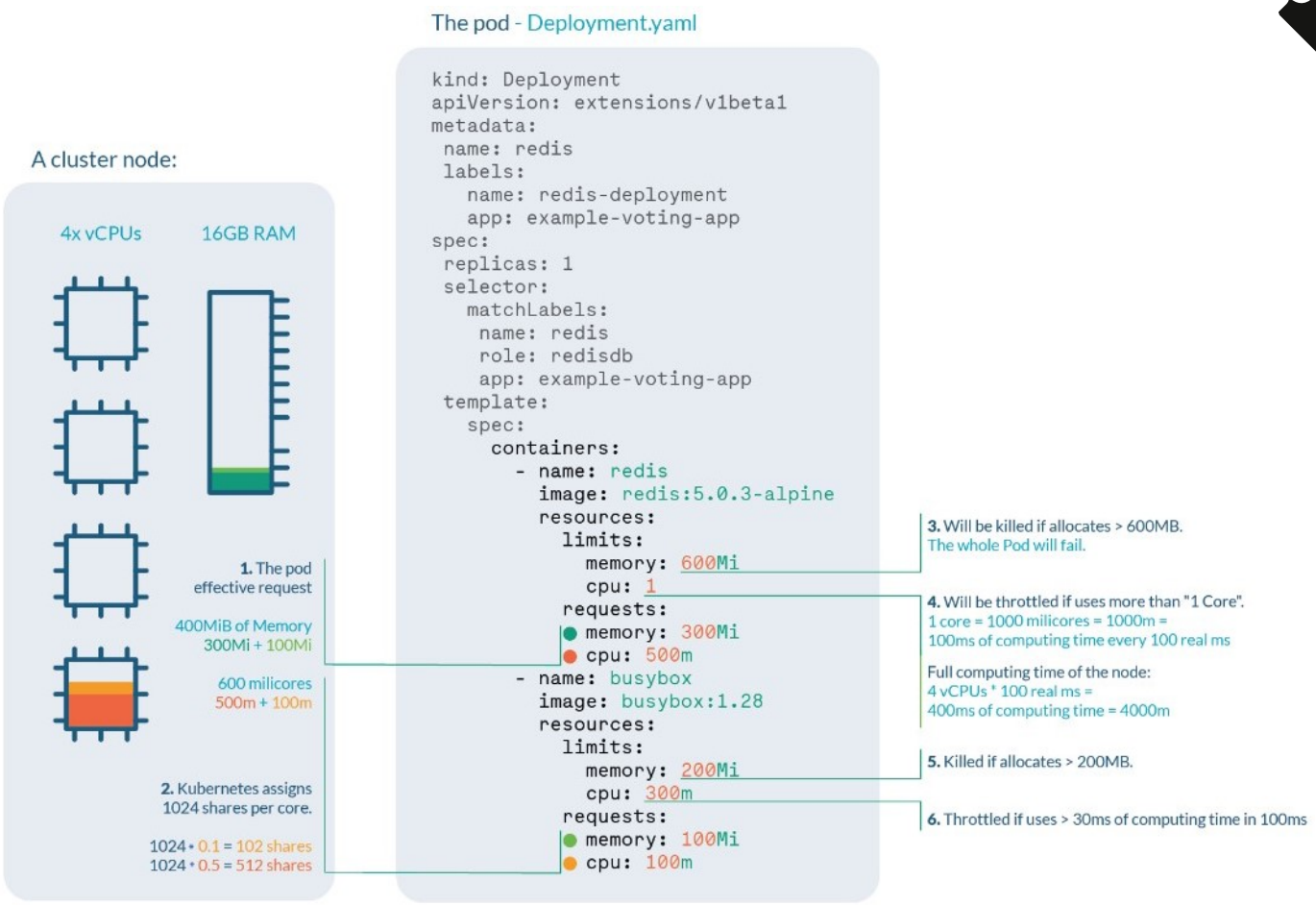
Kubernetes can automatically create a new replica of that pod to continue operations. Pods include one or more containers (such as Docker containers).

Pods also provide environmental dependencies, including persistent storage volumes (storage that is permanent and available to all pods in the cluster) and configuration data needed to run the container(s) within the pod.

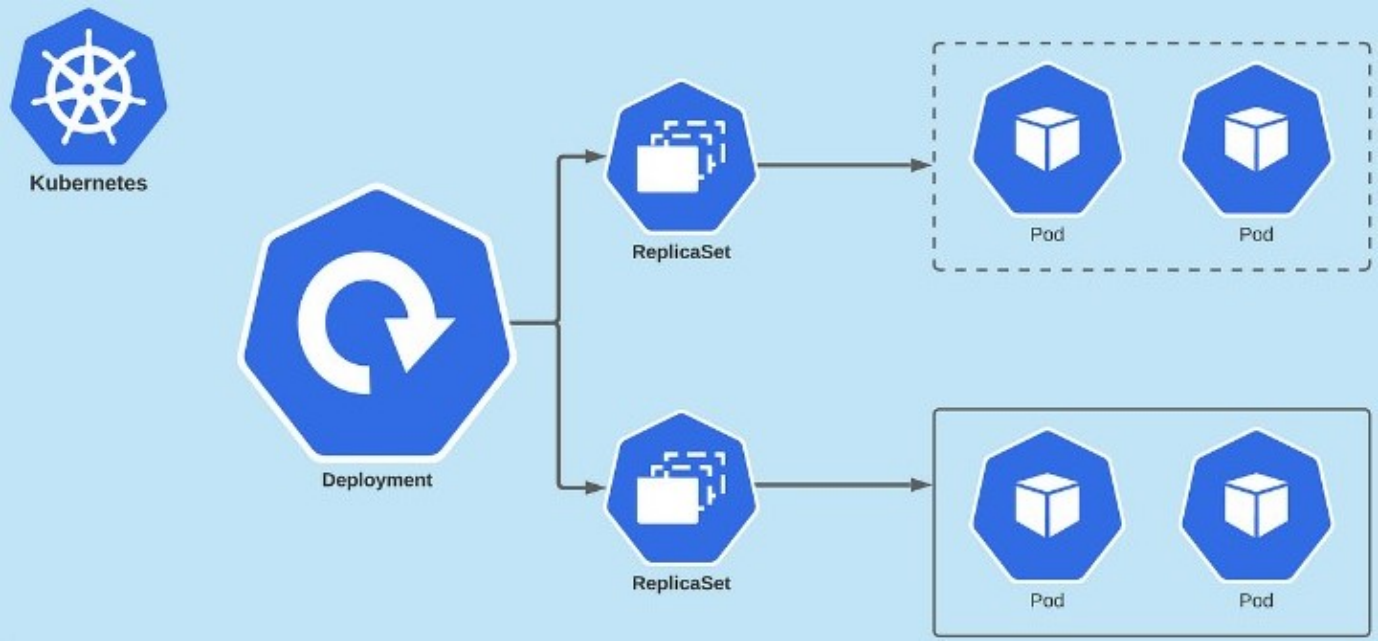
Commands - Pods

```
# create pods without manifest
kubectl run nginx --image nginx
kubectl run -it --rm debug --image=busybox --restart=Never -- sh

# create pod with manifest
kubectl apply -f pod-template.yaml
kubectl create -f pod.yaml

# list pods
kubectl get pods

# list all pods
kubectl get pods --all-namespaces
kubectl get pods -A
kubectl get pods -A -o wide

# list pods in specific node
kubectl get pods --all-namespaces -o wide --field-selector spec.nodeName=worker01

# list pods in  kube-system namespace
kubectl get pod -n kube-system
kubectl get pods -n kube-system -o=jsonpath='{range.items[*]}{"\n"}{.metadata.name}{

# list pods with specif output
kubectl get pods -n kube-system -o yaml
kubectl get pods -n kube-system -o json

# list images used in pods
kubectl get pods -o=jsonpath='{range .items[*]}{"\n"}{.metadata.name}{"\t"}{range .s

# describe details of pods
kubectl describe pod nginx
kubectl -n kube-system  describe pods kube-proxy-worker01

# delete pods
kubectl delete pod nginx
kubectl delete -f pod-template.yaml

# create Service | expose pod
kubectl expose pod my-nginx
```

Understanding Pod Resources



Deployment



A Deployment provides declarative updates for Pods and ReplicaSets. You describe a desired state in a Deployment,and the Deployment Controller
changes the actual state to the desired state at a controlled rate.

Light

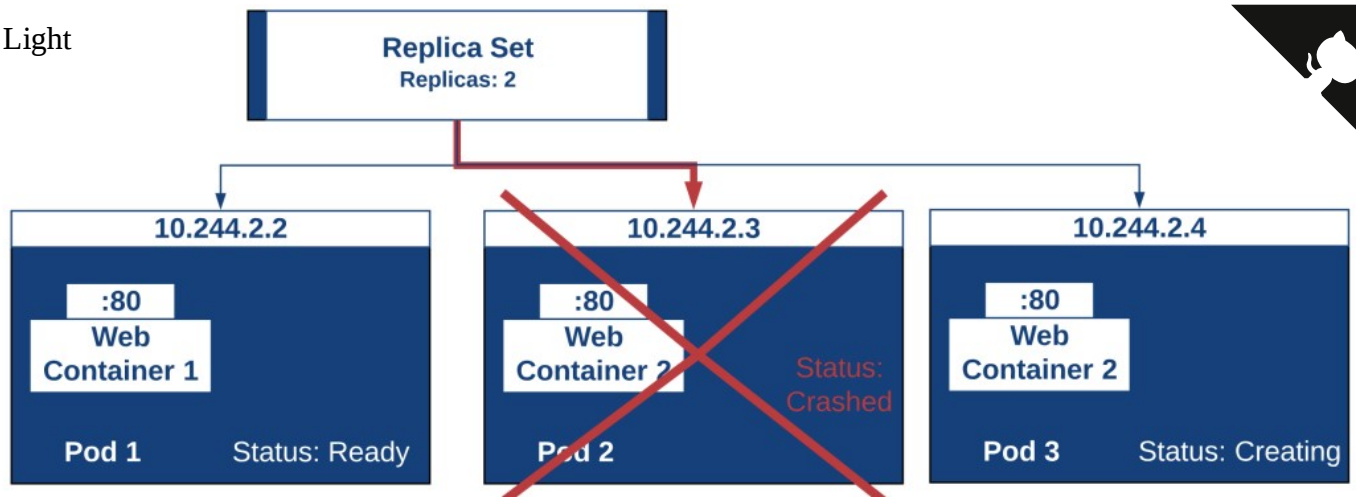
- You can define Deployments to create new ReplicaSets, or to remove existing Deployments and adopt all their resources with new Deployments.

Commands - Deployment

```
# create manifest|template
kubectl run my-nginx  --image nginx --port 80 --dry-run=client -o yaml >pod-template

# apply\update deployment
kubectl apply -f deployment.yaml

# list deployments
kubectl get deployments -A
kubectl -n kube-system get deployments.apps
kubectl get deployments -l app=nginx-deployment

# get pods management by deployment
kubectl get pods -l app=nginx-deployment

# describe deployment
kubectl describe deployment nginx-deployment

# check status of deployment
kubectl rollout status deployment nginx-deployment

# running rollback deployment
kubectl rollout undo deployment nginx-deployment
kubectl rollout undo deployment nginx-deployment --to-revision=1

# get deployment history
kubectl rollout history deployment nginx-deployment
kubectl rollout history deployment nginx-deployment --revision=2

# pause deployment(block updates)
kubectl rollout pause deployment nginx-deployment

# resume deployment(allow updates)
kubectl rollout resume deployment nginx-deployment

# restart deployment (recreate all pods in deployment)
kubectl rollout restart deployment nginx-deployment

# delete deployment
kubectl delete deployment nginx-deployment
```

ReplicaSet

Light



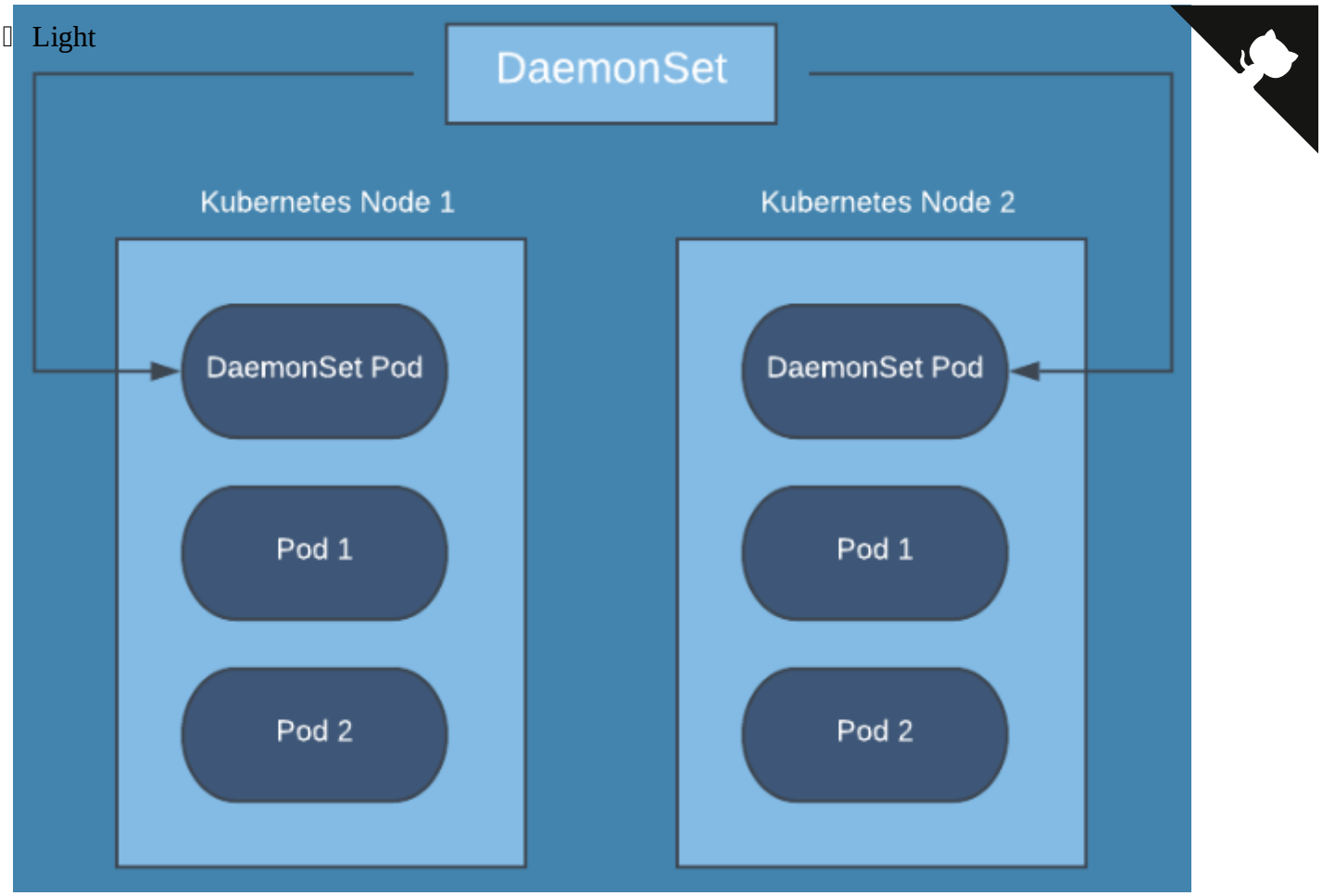
A ReplicaSet's purpose is to maintain a stable set of replica Pods running at any given time. As such, it is often used to guarantee the availability of a specified number of identical Pods.

Commands - ReplicaSet

```
# list replicaset
kubectl get replicaset -l app=nginx-deployment

# describe replicaset
kubectl describe replicaset nginx-replicaset

# create replicaset - see folder replicaset/ for examples
kubectl apply -f replicaset.yaml

# delete replicaset
kubectl delete replicaset nginx-deployment
```

Daemonset

Light

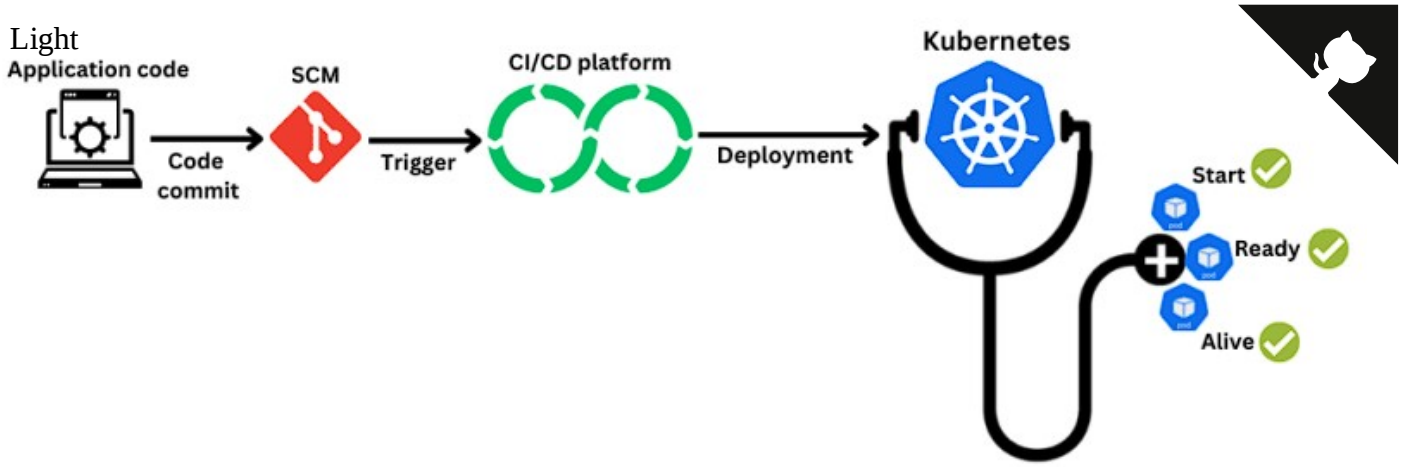


A DaemonSet ensures that all (or some) Nodes run a copy of a Pod.
As nodes are added to the cluster, Pods are added to them.
As nodes are removed from the cluster, those Pods are garbage collected.
Deleting a DaemonSet will clean up the Pods it created.

Commands - Daemonset

```
# list daemonset
kubectl get daemonset -A

#$ describe daemonset
kubectl describe daemonset node-exporter

# create daemonset - see folder daemonset/ for examples
kubectl apply -f daemonset.yaml

# delete daemonset
kubectl delete daemonset node-exporter
```

Probes



Kubernetes probes are health checks that are used to monitor the health of applications and services in a Kubernetes cluster.
Kubernetes probes are typically implemented using the Kubernetes API, which allows them to query the application or service for information.
This information can then be used to determine the application's or service's health.
Kubernetes probes can also be used to detect changes in the application or\ service and send a notification to the Kubernetes control plane, which can\ then take corrective action.

Types of Probes

Startup Probes

A startup probe is used to determine if a container has started successfully.
This type of probe is typically used for applications that take longer to start up,
or for containers that perform initialization tasks before they become ready to receive traffic.
The startup probe is run only once, after the container has been created, and it will delay the start of the readiness and liveness probes until it succeeds.
If the startup probe fails, the container is considered to have failed to start
and Kubernetes will attempt to restart the container.

Readiness Probes

A readiness probe is used to determine if a container is ready to receive traffic.
This type of probe is used to ensure that a container is fully up and running and can
accept incoming connections before it is added to the service load balancer.
A readiness probe can be used to check the availability of an application's dependencies
or to perform any other check that indicates the container is ready to serve traffic.
If the readiness probe fails, the container is removed from the service load balancer until the probe succeeds again.

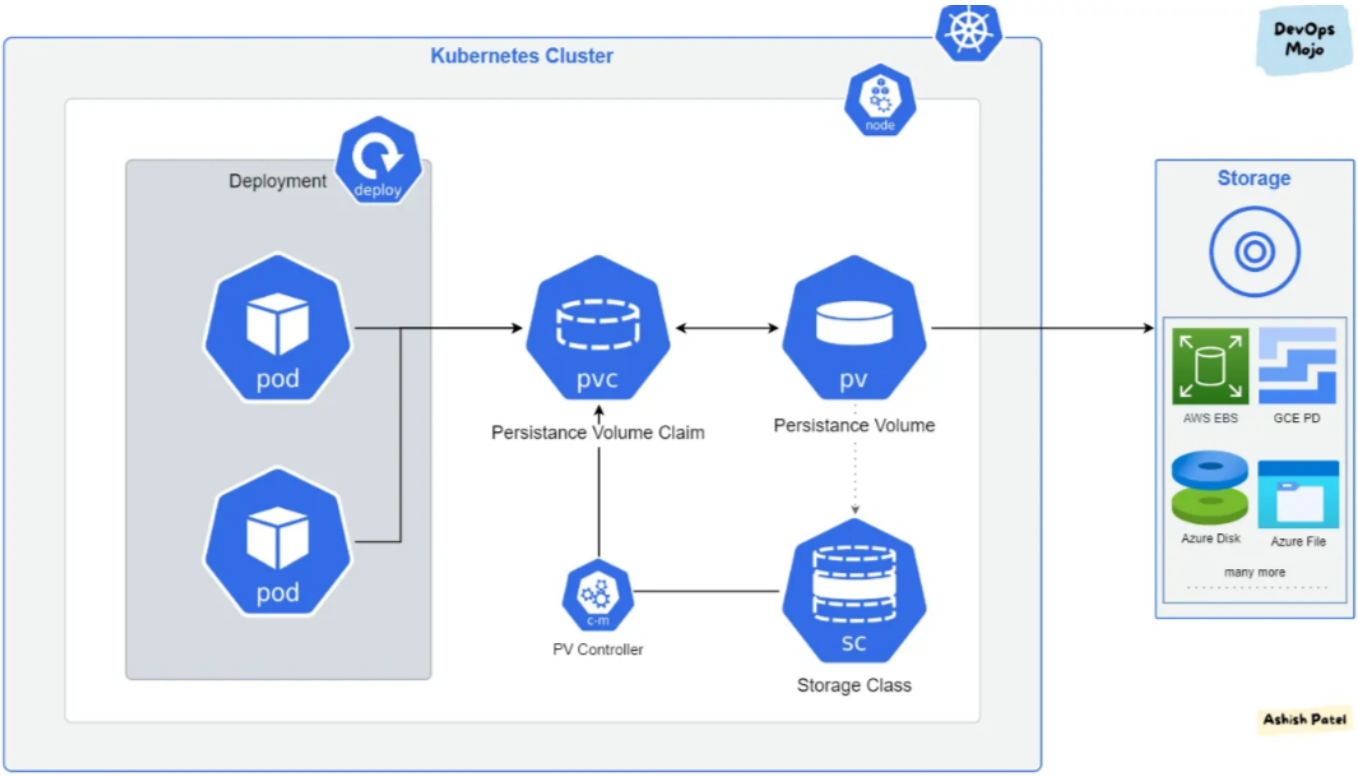
Liveness Probes

A liveness probe is used to determine if a container is still running and functioning properly.
This type of probe is used to detect and recover from container crashes or hang-ups.

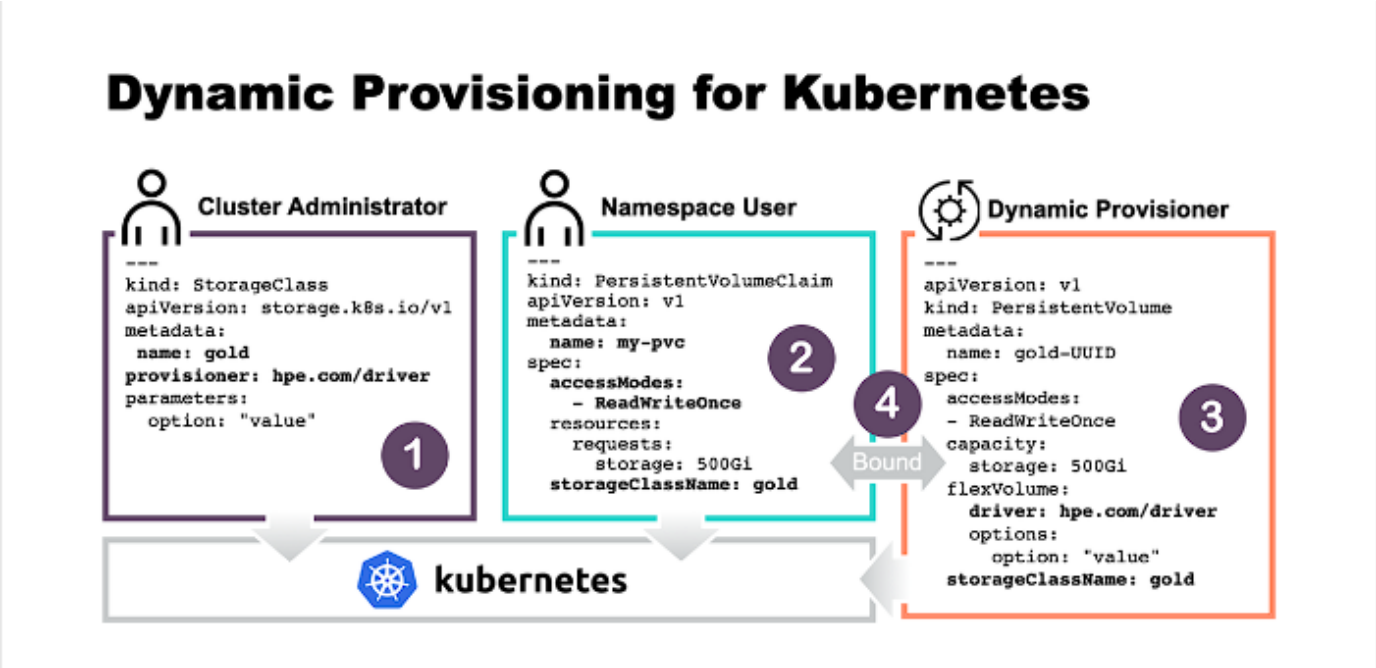
Light

A liveness probe can be used to check the responsiveness of an application or to perform any other check that indicates the container is still alive and healthy.
If the liveness probe fails, Kubernetes will attempt to restart the container to restore its functionality.

Volumes



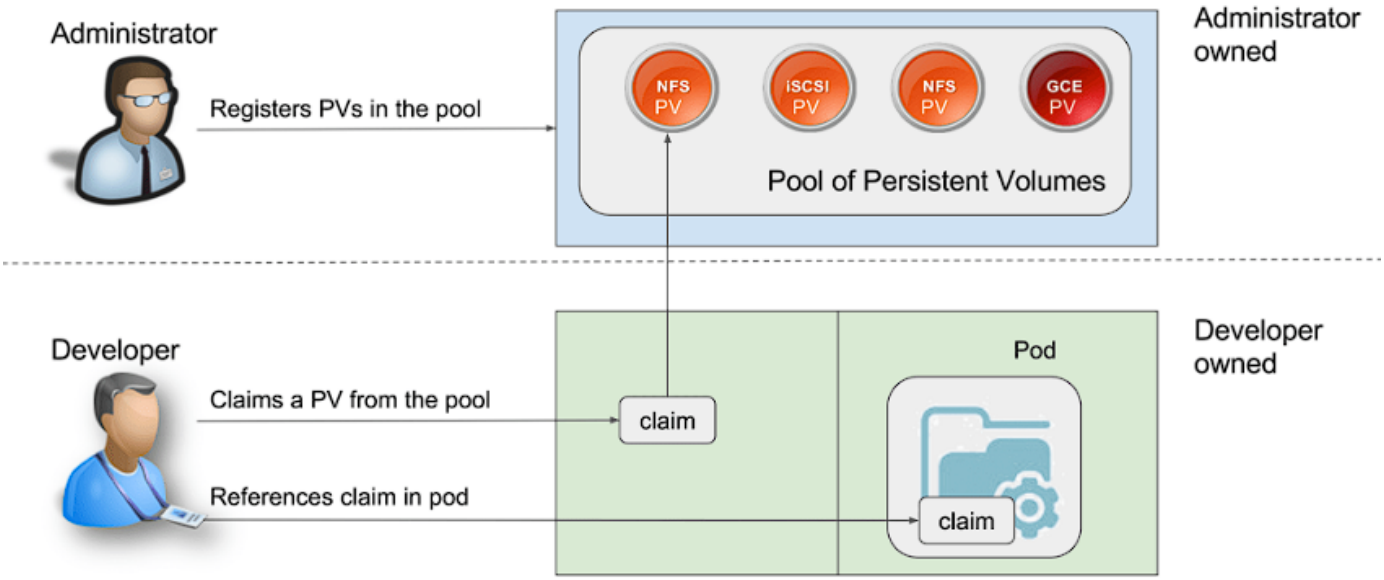
Storage Class



Some Storage Class providers

- kubernetes.io/aws-ebs: AWS Elastic Block Store (EBS)
- kubernetes.io/azure-disk: Azure Disk
- kubernetes.io/gce-pd: Google Compute Engine (GCE) Persistent Disk
- kubernetes.io/cinder: OpenStack Cinder
- kubernetes.io/vsphere-volume: vSphere
- kubernetes.io/no-provisioner: Volumes locais
- kubernetes.io/host-path: Volumes locais

PV - Persistent volume



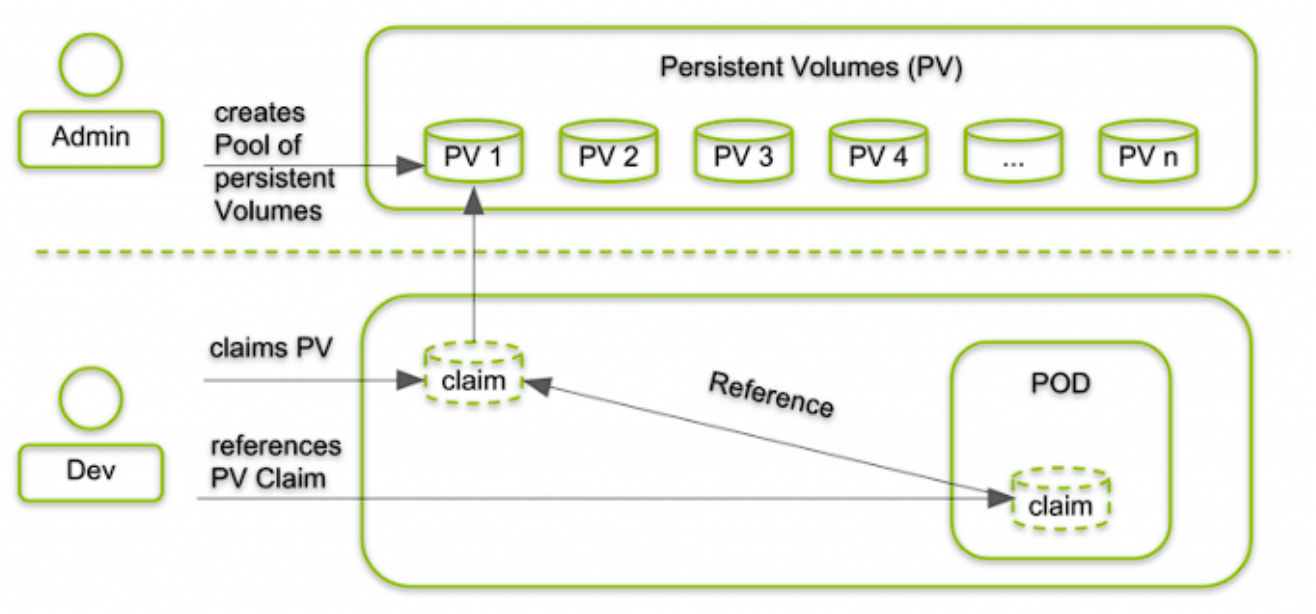
Some Types of PV

- Local
  - HostPath
- Network
  - NFS
  - iSCSI
  - Ceph RBD(RAdos Block Device)
  - GlusterFS
  - Cloud Providers(EBS,Google Cloud Persistent Disk,Azure Disk Storage)

Type of Storage(hostPath)

- hostPath
- nfs
- iscsi
- csi
- local

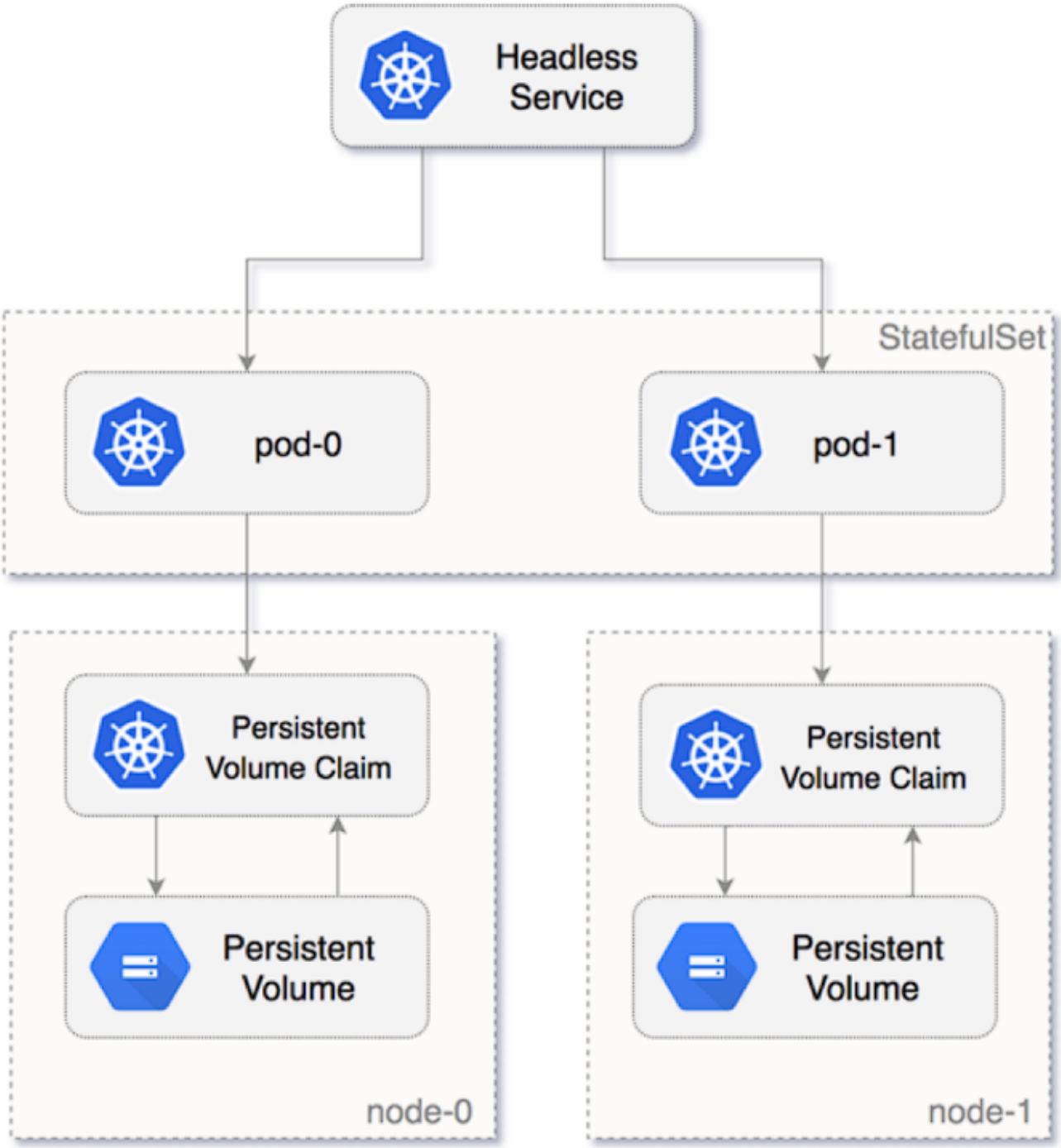
PVC - Persistent Volume Claim



Commands - Volumes

```
# list storage classes
kubectl get storageclass

# describe storage class
kubectl describe storageclass silvestrini

# get storage class provisioners
kubectl get storageclasses.storage.k8s.io  -o=jsonpath='{range.items[*]}{.provisione

# list pv in cluster
kubectl get pv -A

# describe pv
kubectl describe pv my-pv

# list pvc
kubectl get pvc -o wide

# delete pvc
kubectl delete pvc my-pvc

# describe pvc
kubectl describe pvc my-pvc

# get events
kubectl get events my-pvc.1772cda2d4c7069b
```

StatefulSet



StatefulSet is the workload API object used to manage stateful applications.

Manages the deployment and scaling of a set of Pods, and provides guarantees about the ordering and uniqueness of these Pods.

Like a Deployment, a StatefulSet manages Pods that are based on an identical container spec. Unlike a Deployment, a StatefulSet maintains a sticky identity for each of its Pods.\

These pods are created from the same spec, but are not interchangeable: each has a persistent identifier that it maintains across any rescheduling.

Light

If you want to use storage volumes to provide persistence for your workload, you can use a StatefulSet as part of the solution. Although individual Pods in a StatefulSet are susceptible to failure, the persistent Pod identifiers make it easier to match existing volumes to the new Pods that replace any that have failed.

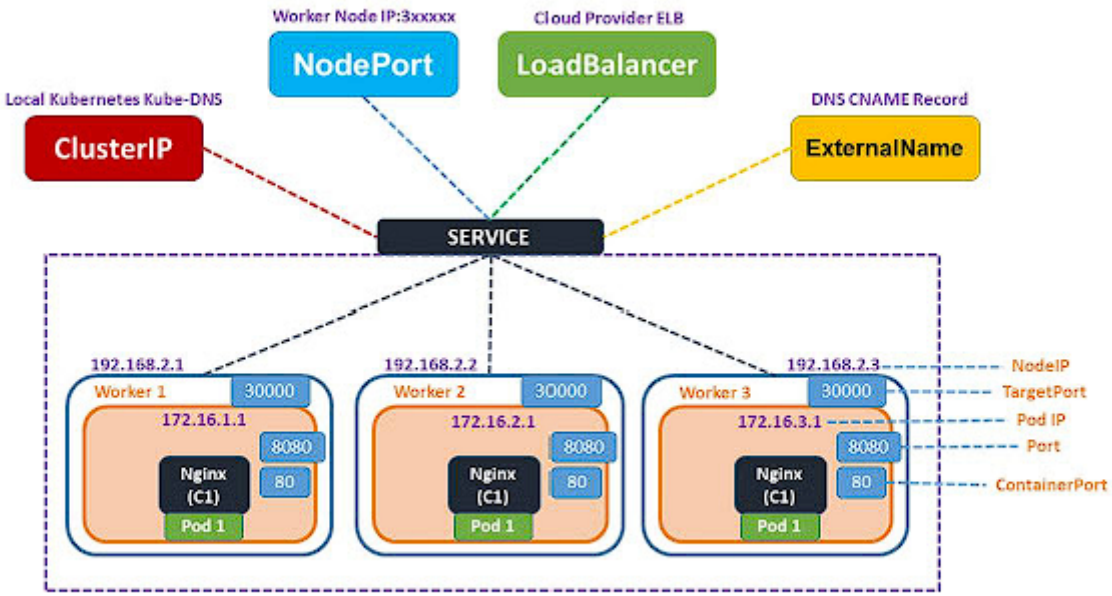
DNS for pods in StatefulSet

```
<pod-name>.<service-name>.<namespace>.svc.cluster.local
# Example: nginx-0.nginx.default.svc.cluster.local
```

Commands - StatefulSet

```
# list statefulsets
kubectl get statefulsets

# describe statefulsets
kubectl describe  statefulsets.apps  nginx

# delete statefulset
kubectl delete statefulset nginx

# test network service
## create container for test
kubectl run -it --rm --image=busybox --restart=Never -- sh

## test nslookup
nslookup  nginx-0.nginx.default.svc.cluster.local

## test web page
wget -O- http://nginx-0.nginx.default.svc.cluster.local
```

Services



A Kubernetes service is a logical abstraction for a deployed group of pods in a cluster (which all perform the same function).
Since pods are ephemeral, a service enables a group of pods, which provide specific functions (web services, image processing, etc.) to be assigned a name and unique IP address (clusterIP).

Service Types in Kubernetes

ClusterIP Services

ClusterIP is the default service type in Kubernetes, and it provides internal connectivity between different components of our application. Kubernetes assigns a virtual IP address to a ClusterIP service that can solely be accessed from within the cluster during its creation. This IP address is stable and doesn’t change even if the pods behind the service are rescheduled or replaced.

ClusterIP services are an excellent choice for internal communication between different components of our application that don’t need to be exposed to the outside world. For example, if we have a microservice that processes data and sends it to another microservice for further processing, we can use a ClusterIP service to connect them.

To create a ClusterIP service in Kubernetes, we need to define it in a YAML file and apply it to the cluster. Here's an example of a simple ClusterIP service definition:

```
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend
```

```
  ports:
  - name: http
    port: 80
    targetPort: 8080
```

In this example, we define a service named backend with a selector that targets pods labeled with app: backend. The service exposes port 80, which is the port used by clients to access the service, and forwards the traffic to the pods' port 8080, which is where the backend application is running.

NodePort Services

NodePort services extend the functionality of ClusterIP services by enabling external connectivity to our application. When we create a NodePort service on any node within the cluster that meets the defined criteria, Kubernetes opens up a designated port that forwards traffic to the corresponding ClusterIP service running on the node.

These services are ideal for applications that need to be accessible from outside the cluster, such as web applications or APIs. With NodePort services, we can access our application using the node's IP address and the port number assigned to the service.

Let's look at an example of a simple NodePort service definition:

```
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector:
    app: frontend
  type: NodePort
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

We define a service named frontend that targets pods labeled with app: frontend by setting a selector. The service exposes port 80 and forwards the traffic to the pods' port 8080. We set the service type to NodePort, and Kubernetes exposes the service on a specific port on a qualifying node within the cluster.

When we create a NodePort service, Kubernetes assigns a port number from a predefined range of 30000-32767. Additionally, we can specify a custom port number by adding the nodePort field to the service definition:

```
apiVersion: v1
```

```
kind: Service
metadata:
  name: frontend
spec:
  selector:
    app: frontend
  type: NodePort
  ports:
  - name: http
    port: 80
    targetPort: 8080
    nodePort: 30080
```

The nodePort field is specified as 30080, which tells Kubernetes to expose the service on port 30080 on every node in the cluster.

LoadBalancer Services

LoadBalancer services connect our applications externally, and production environments use them where high availability and scalability are critical. When we create a LoadBalancer service, Kubernetes provisions a load balancer in our cloud environment and forwards the traffic to the nodes running the service.

LoadBalancer services are ideal for applications that need to handle high traffic volumes, such as web applications or APIs. With LoadBalancer services, we can access our application using a single IP address assigned to the load balancer.

Here's an example of a simple LoadBalancer service definition:

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  type: LoadBalancer
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

We set the service type to LoadBalancer to instruct Kubernetes to provision a load balancer. Here, we define a service named web and specify a selector that targets pods labeled with app: web. Additionally, we expose port 80 and forward traffic to the pods' port 8080.

After creating the LoadBalancer service, Kubernetes provisions a load balancer in the cloud environment with a public IP address. We can use this IP address to access our application from outside the cluster.

Light

Service Type	Use Case	Accessibility	Resource Allocation
ClusterIP	Internal communication between application components	Within the cluster only	Minimal resources needed
NodePort	External accessibility for web applications or APIs	Accessible from outside the cluster via a high-numbered port on the node	Additional resources needed
LoadBalancer	Production environments with high traffic volumes	Accessible from outside the cluster via a load balancer	Significant resources needed
Cloud Provider's Load Balancer	Using a cloud provider for Kubernetes	Accessible from outside the cluster via the cloud provider's load balancer	May result in cost savings and better performance

Reference: https://www.baeldung.com/ops/kubernetes-service-types

Commands - Services

```
# list services
kubectl get services
kubectl get svc -o wide

# list services in system namespace
kubectl get svc -n kube-system

# details of services
kubectl describe svc nginx
kubectl describe svc -n kube-system
```

```
# list endpoints
kubectl get endpoints

# create ClusterIP service
kubectl expose deployment app-silvestrini --port=80 --target-port=8080

# create NodePort service
kubectl expose deployment app-silvestrini --type=NodePort --port=80 --target-port=80

# create loadbalance service
kubectl expose deployment app-silvestrini --type=LoadBalancer --port=80 --target-por

#create external name service
kubectl create service externalname app-silvestrini --external-name my-db.skynet.com

# delete service
kubectl delete service nginx
```

(back to services)

(back to top)

Contributing

Contributions are what make the open source community such an amazing place to learn, inspire, and create. Any contributions you make are **greatly appreciated**.

If you have a suggestion that would make this better, please fork the repo and create a pull request. You can also simply open an issue with the tag "enhancement". Don't forget to give the project a star! Thanks again!

1. Fork the Project
2. Create your Feature Branch (`git checkout -b feature/AmazingFeature`)
3. Commit your Changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the Branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

License

- This project is licensed under the MIT License * see the LICENSE.md file for details

Contact

Marcos Silvestrini - marcos.silvestrini@gmail.com

Follow @mrsilvestrini

(back to top)

Acknowledgments

- CNCF - Cloud Native Computing Foundation
- OCI - Open Container Initiative
- Borg
- kubernetes Website
- Github
- Issues
- CKA Certification
- CKAD Certification
- CKS Certification
- Book Linuxips
- Kind
- Minikube
- k0s
- k3s
- RKE2
- Up RKE2 Cluster HA
- Kubernetes Architecture
- Rancher Local Path Provisioner
- Kubernetes Dashboard

(back to top)